

CADFA: A Clock Skew-Based Active Device Fingerprint Authentication Scheme for Class-1 IoT Devices

Chao Shang¹, Jin Cao¹, *Member, IEEE*, Tong Zhu, Yinghui Zhang², *Member, IEEE*, Ben Niu³, *Member, IEEE*, and Hui Li¹, *Member, IEEE*

Abstract—The Class-1 Internet of Things (IoT) device is difficult to protect because of the lack of minimal hardware and extension support. Researchers have proposed several remote attestation schemes to protect IoT devices. However, hardware-assisted remote attestation methods require the support of minimal hardware, which is not suitable for Class-1 IoT devices. And regular software-based methods can only verify the integrity rather than the identity of devices. In this article, we propose a new identity authentication scheme for Class-1 IoT devices based on clock skew, achieving not only identity authentication but also device integrity verification. The formal verification results by employing the Tamarin tool show that our protocol achieves mutual authentication, man-in-the-middle attack resistance, replay attack resistance, and counterfeit attack resistance. And the proposed authentication protocol is evaluated on a small cluster of Arduino Mega boards, achieving 100% accuracy of identification. Our work combines remote attestation and device fingerprint and is fit for Class-1 IoT devices.

Index Terms—Authentication, Class-1 IoT device, device fingerprint, remote attestation.

NOMENCLATURE

Entity	Description
P	Proof node.
V	Verification node.
Device Argument	Description
ID_i	Identification of device i .
EK_i	Encryption key of device i .
IK_i	Integrity key of device i .
Typ_n	Device model.

$Freq_n$	Processor frequency.
SF_n	FLASH memory size.
SR_n	RAM size.
Protocol Argument	Description
$Hash()$	Hash function.
$ENC()$	Encryption operation.
$HMAC()$	Hash-based message authentication code.
R_i	Random number.
SEQ	Sequence number.
$Checksum$	Verification and response checksum.
N	Number of <i>checksum</i> .
t	Time interval of clock skew authentication.
Tv_i	Current time of verification node i th authentication.
Tp_i	Current time of proof node i th authentication.
c_θ	Number of clock skew-based authentication requests.
θ_1	Threshold value of the difference between the clock skew and the previous skew.
θ_2	Threshold value of the difference between the recorded clock skew and the current skew.
C_n	Total number of possible pairs (i, j) where $i \neq j$.

I. INTRODUCTION

WITH the development of Internet of Things (IoT) technology over decades, IoT devices have evolved from large size to microchip-based intelligent electric devices. Particularly in recent years, with the urgency of microchips and various types of sensors, much attention has been paid to IoT devices within which, the Class-1 IoT devices [1] are especially focused because of their flexibility and low cost. According to the Internet Engineering Task Force (IETF) document RFC 7228, Class-1 IoT devices refer to devices whose data size [e.g., random access memory (RAM)] is less than 10 KB and code size (e.g., flash) is less than 100 KB. Due to the poor capability, full protocol stacks (e.g., hypertext transfer protocol, transport layer security, and related security protocols) cannot be easily employed [2] to the Class-1 IoT devices. As a result, Class-1 IoT

Manuscript received 16 May 2023; revised 1 September 2023; accepted 2 January 2024. Date of publication 25 January 2024; date of current version 15 March 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U23B2024, Grant 62172317, and Grant 62102298, and in part by the China Scholarship Council. (*Corresponding author: Jin Cao.*)

Chao Shang, Jin Cao, Tong Zhu, and Hui Li are with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xian 710126, China (e-mail: chao.shang@stu.xidian.edu.cn; caoj897@gmail.com; tongzhu@stu.xidian.edu.cn; lihui@mail.xidian.edu.cn).

Yinghui Zhang is with the National Engineering Laboratory for Wireless Security, Xi'an University of Posts & Telecommunications, Xi'an 710049, China (e-mail: yhzhaang@163.com).

Ben Niu is with the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100045, China (e-mail: niuben@iie.ac.cn).

Digital Object Identifier 10.1109/JSYST.2024.3351222

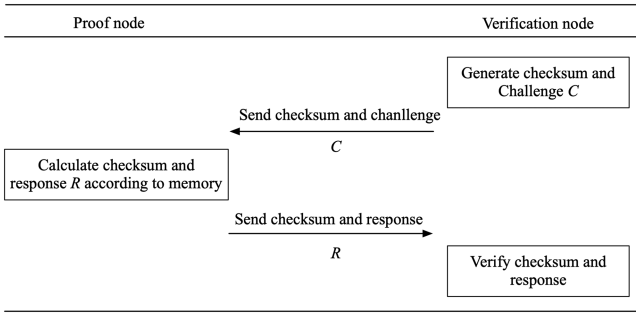


Fig. 1. Typical remote attestation procedure.

devices are facing serious security risks. To protect these kinds of devices, the remote attestation method is adopted by researchers because of its scalability and easy implementation. Next, we will introduce remote attestation, and device fingerprint, respectively, and state the current problem and our contributions.

A. Remote Attestation

A typical remote attestation procedure is demonstrated in Fig. 1. As shown in Fig. 1, a device about to be proved is requested by a trusted verifier to remotely send a proof message to guarantee its integrity or other security features. In general, there are three kinds of remote attestation (RA) schemes as follows: 1) hardware based, 2) software based, and 3) a combination of both. The hardware-based method usually uses a trust hardware root to perform the attestation stage [3], [4]. It usually needs the prover to support at least the minimum hardware functionality, such as read-only memory (ROM) and memory protection unit (MPU) [5]. However, it is difficult for the Class-1 devices to conduct hardware-based attestations, because most of them do not support minimal hardware features (for example, MICAz and MicroPnP do not contain ROM, and Arduino Nano, Arduino Micro, and Arduino UNO do not support MPU [6], [7]). The software-based scheme relies on strict time control to finish the attestation [8], [9]. Cao et al. [10] have proposed a software-based RA scheme to protect Class-1 IoT devices' software integrity in. However, device identity authentication cannot be accomplished in the proposed method. Without hardware assurance, the key set in the register stage is not secure enough in front of extreme situations. This scheme in [10] presents a device fingerprint-based scheme to achieve device identity authentication, using clock skew to guarantee security under physical attacks.

B. Device Fingerprint

Device fingerprint is generated by extracting the inherent properties of the device's hardware and software. The subtle differences in the device components make it difficult to fabricate. Thus, it can be a unique identifier to distinguish each device. Similar to biometric fingerprints with different patterns, device fingerprint also has its own unique feature sets. Generally speaking, different kinds of devices will have different features. Hardware features include RF features, clock skew, and physical

unclonable function (PUF) [11] component features, etc.; software features include the operating system, software behavior, communication traffic, etc.

Researchers have been studying authentication methods based on device fingerprints, but most existing approaches heavily rely on feature sets and machine learning techniques. In addition, to get satisfactory results, the feature extracting and prepossessing stage is complicated and burdensome, let alone the time-consuming and data-consuming training stage.

To avoid the heavy burden due to the use of machine learning to extract and train, single features with high discriminative ability have attracted much attention, such as clock skew. Usually, a digital device clock signal is produced by a crystal oscillator. Even under the same type and the same series, the produced clock will have slight variations because of the mechanical accuracy during the oscillator production.

C. Problem Statement

The current hardware-assisted remote attestation methods require the prover to support at least minimal hardware features, which Class-1 IoT devices do not have [5]. Besides, the software-based attestation methods can only provide device integrity, and cannot achieve identity authentication [10]. Some researchers have proposed using device fingerprint to identify a device, but most use machine learning techniques, requiring much training time and large datasets [12]. Although attestation methods combined with a single device fingerprint have been proposed, most of them need minimal hardware features like ROM and MPU [13], which is not suitable for Class-1 IoT devices.

D. Contribution

In this article, we proposed a clock skew-based active device fingerprint authentication protocol (CADFA) for Class-1 IoT devices. Our contributions are described as follows.

- 1) We combine the remote attestation protocol in [10] and clock skew-based device fingerprint, and thus, propose a new identity authentication scheme, achieving not only identity authentication but also integrity verification.
- 2) We verify the proposed authentication protocol using the protocol formal verification tool Tamarin. The verification results show our protocol supports security properties, including mutual authentication, physical attack resistance, man-in-the-middle attack resistance, and replay attack resistance.
- 3) We evaluate our system on a small cluster of IoT devices. The result indicates that our protocol performs well on Class-1 IoT devices.

The rest of this article is organized as follows. Section II presents the related work. Section III introduces the clock skew attribute. And Section IV describes the system model and security requirements. The detail of the proposed scheme is introduced in Section V. Section VI and VII give the security analysis and performance evaluation, respectively. Subsequently, we simulate the proposed scheme using the Arduino boards in Section VIII. Finally, Section IX concludes this article.

TABLE I
COMPARISON OF DEVICE FINGERPRINT RESEARCH

	Zuo [16]	Sanchez [29]	Lanze [27]	Zhou [33]	Lorenz [30]	Ibrahim [13]
Attribute-based	Yes	No	No	No	No	Yes
Hardware-based	No	Yes	Yes	Yes	Yes	Yes
Additional sensors	No	No	No	Yes	Yes	No
High precise clock	No	Yes	Yes	No	No	No
Accurate timing	No	Yes	No	No	No	No
Class-1 IoT devices	Yes	No	Yes	No	No	No

II. RELATED WORK

Currently, there are two catalogs of device fingerprint authentication schemes as follows: 1) attribute-based and 2) hardware-based. Attribute-based device fingerprinting relies on device setting attributes. Eckersley et al. [14] proposed to use multiple browser attributes as device fingerprints, such as browser-installed fonts, plug-ins, screen size, resolution, etc. However, Vastel et al. [15] pointed out that these attributes usually change quickly and expire in less than ten days. In addition, these attributes are extremely easy to be forged. Zuo et al. [16] used the permanent device index (UUID) in low-power Bluetooth (BLE) as the device fingerprint. They extracted the UUID by analyzing the applications paired with BLE and inferred the device type and functional usage.

Hardware-based device fingerprint is initially used for authentication. The current features employed include the manufacturer difference [17], [18], [19], PUF [20], [21], [22], and the embedded sensors deviation [23].

Among the hardware features, the clock skew draws a lot of attention. Kohno et al. [24] used frame timestamps to identify PCs. Jana and Kasera [25] used timestamps of beacon frames on wireless LANs to identify unauthorized wireless access points. Sharma et al. [26] proposed to use frame headers to detect clock skew. They tested with 210 different devices which can be distinguished between different kinds of devices and the same kind of devices. In addition, they tested the stability of the clock skew under different measurement methods and environmental factors (e.g., temperature or operating system), and demonstrated that the method applies to medium-sized networks. Lanze et al. [27] proposed a clock skew-based device fingerprinting algorithm that exploits the timestamps of beacon frames sent periodically by a wireless access point and provided a termination criterion that can eliminate the deviation of the measurement device. Radhakrishnan et al. [28] proposed a clock skew-based fingerprinting system called GTID, which is suitable for small-scale networks.

API can also be used to extract device fingerprints. Sanchez et al. [29] proposed a time-based device fingerprinting technique to identify computer devices by timing a few lines of JavaScript code and implemented a tool for distinguishing computer devices with identical software and hardware. Although efficient, it requires high temporal accuracy and accurate timing capabilities. Lorenz et al. [30] proposed to use IoT sensors embedded circuits as fingerprints. They provided predefined voltage sequences to sensors and distinguished them by monitoring the output sequences.

Also, there are works using electromagnetic signals as device fingerprints to distinguish a specific device from the same model.

Jafari et al. [31] used deep learning-based methods to distinguish devices. They tested six identical ZigBee devices and concluded that the devices could be identified based on their RF traces. Cheng et al. [32] proposed to identify identical laptop and smart-phone devices based on CPU signals, but the solution requires an external sensor to capture the signals within a 16-mm distance. Zhou et al. [33] proposed a robust RF fingerprinting scheme, addressing the performance unreliability, channel fading interference, and unauthorized device authentication problems in RF fingerprinting studies.

Researchers have combined clock skew and remote attestation methods [11], [13], [34], [35], [36], but most of the schemes require the devices to have ROM and MPU or PUF, which is not suitable for Class-1 IoT devices.

In Table I, we select typical solutions in various fields for comparison. The dimensions include the solution type, the sensor requirement, the precision requirement (e.g., accurate to microseconds), the time accuracy requirement, and most importantly, the Class-1 IoT devices' applicability. As shown in Table I, attribute-based device fingerprinting schemes usually use certain parameters of the device as fingerprints and do not require additional hardware requirements. Thus, they have the widest range of applicability but are highly susceptible to forgery and expire quickly, making them unsuitable for authentication. The API-based solutions require accurate timing ability but Class-1 IoT devices such as the Arduino UNO cannot achieve accurate timing. Clock skew-based solutions only need a high-precision clock, suitable for Class-1 IoT devices. Electromagnetic signal-based solutions require additional sensor devices for acquisition and are also not suitable for Class-1 IoT devices. The RA methods combined with clock skew require the prover to support PUF, ROM, or MPU, which cannot apply to Class-1 IoT devices.

III. CLOCK SKEW ATTRIBUTE

In this section, we introduce where the skew comes from and how to measure it. To provide current time $T_c(t)$, a device clock c calculates the time step from an initial point of the real-time i_c , which can be represented as $T_c(t) = t - i_c$. However, because of the oscillator difference, the standard hardware clock c is not accurate and has an offset

$$off_c(t) = R_c(t) - T_c(t)$$

where $T_c(t)$ is the provided time and $R_c(t)$ is the true time. Clock skew is the first-order derivative of its offset. Since the clock skew is constant throughout a measurement, the skew in the rather small time window between t_1 and t_2 can be

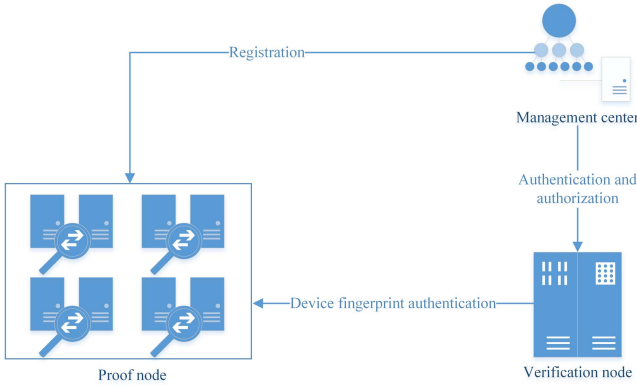


Fig. 2. Clock skew-based active device fingerprint authentication system.

calculated by

$$s_c[t_1, t_2] = \frac{off_c(t_2) - off_c(t_1)}{t_2 - t_1}.$$

In this article, we use the scheme in [27] to obtain the true time of the tested devices.

IV. SYSTEM MODEL AND SECURITY REQUIREMENT

A. System Model

As shown in Fig. 2, the proposed clock skew-based active device fingerprint authentication system consists of a management center, verification nodes, and proof nodes. The management center is the network manager, responsible for registering nodes, distributing keys, and verifying node authorization. It is considered equipped with abundant computational resources and sufficient storage resources, as well as trustful hardware and strong self-protection ability. In the same way that the management center presented in [10], it defaults to a trusted device in this protocol. The verification node is the launcher of the authentication, similar to the management center, it carries abundant resources, secure hardware, and high time accuracy. Same as [10], it is considered a trusted device in this protocol. The proof node is regarded as a resource-limited device like Class-1 IoT devices. It is untrusted in this protocol.

B. Security Requirement

As mentioned above, the management center is the manager of the network, responsible for the registration and authorization with trustful hardware. Thus, the privacy data can be transmitted through the secure channel between the management center and verification nodes. In general, the system supports the following security features.

- 1) *Mutual authentication*: To guarantee the identity and message legitimacy of the two parties, they need to authenticate each other during the protocol.
- 2) *Physical-attack resistance*: The Class-1 IoT devices do not support built-in secure hardware, in extreme situations, the attacker could compromise the keys and data in the original devices and replace them with malicious devices. In this case, the proposed protocol is supposed to resist physical attack and ensure the original device's existence.

- 3) *Several protocol attacks resistance*: Besides, the proposed scheme can defend against man-in-the-middle attacks, replay attacks, and counterfeit attacks.

V. PROPOSED SCHEME: CADFA

Typically, in the existing device fingerprinting methods, the verification node passively collects timestamps in the messages periodically sent from the proof node. The accuracy of clock data is not ensured since the existence of verification deviation. And the time interval between two messages is wasted because of the passive waiting mechanism. In this article, we present a clock skew-based active device fingerprint authentication protocol. Compared with the existing schemes, we guarantee the clock skew accuracy by mathematics processing and propose an active mechanism to shorten the passive waiting time interval.

The proposed protocol CADFA consists of the following two stages: 1) the registration stage and 2) the authentication stage. The registration stage distributes identifiers and keys for new nodes and allocates device information as part of the device fingerprint for further authentication. In the authentication stage, the verification node will initiate the device fingerprint authentication for Class-1 IoT devices and judge a device's legitimacy according to the collected information during the authentication. A list of frequently used notations is provided in Nomenclature.

A. Registration Stage

When a new node n joins the network, a master key $K = EK || IK$ and a node identifier ID_n are preset by the management center, where EK is used to encryption and IK is the integrity key. Meanwhile, the management center will collect the device information, including device model Typ_n , processor frequency $Freq_n$, FLASH memory size SF_n , and RAM size SR_n as a part of the device fingerprint. The other part will be collected in the further authentication process, from which the clock skew is acquired for the subsequent authentication, we will discuss it later.

B. Authentication Stage

In this stage, there are two parts of the device fingerprint used in this stage. One is the basic device information that we have mentioned before, which is directly sent by the proof node after generation and calculation. The other is clock skew, which will be sent in a challenge message. The detailed protocol is described as shown in Fig. 3.

- Step 1*: The verification node V selects a random number R_1 , encrypts it with EK to derive $C_1 = Enc_{EK}(R_1)$ and computes $mac_1 = HMAC_{IK}(C_1 || ID_v || ID_p)$ by using IK .
- Step 2*: Then, V sends $\langle ID_p, ID_v, C_1, mac_1 \rangle$ to the proof node P .
- Step 3*: Upon receiving the authentication request message, P checks mac_1 and decrypts C_1 using EK and acquires R_1 . Then, P generates a random number R_2 and encrypts it with EK . P calculates $C_2 = Enc_{EK}(R_2)$ and $mac_2 = HMAC_{IK}(C_2 || ID_v || ID_p)$.

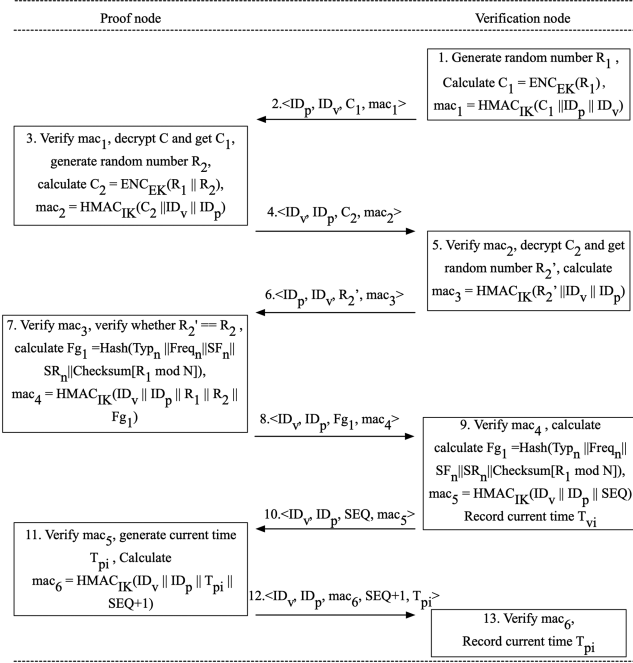


Fig. 3. Authentication stage.

- Step 4:** Then, P sends $\langle ID_p, ID_v, C_2, mac_2 \rangle$ to V .
- Step 5:** Upon receiving the authentication response message, V verifies mac_2 and decrypts C_2 to acquire R'_2 . Then, V calculates $mac_3 = HMAC_{IK}(R'_2 || ID_v || ID_p)$.
- Step 6:** Then, V sends $\langle ID_p, ID_v, R'_2, mac_3 \rangle$ to P .
- Step 7:** Upon receiving the message, P first verifies mac_3 and checks whether $R_2 = R'_2$. If true, the node P has authenticated the EK and IK of V . Then, P collects its device model Typ_n , processor frequency $Freq_n$, FLASH Memory size SF_n , and RAM size SR_n . After that, according to the *Checksum* and *Checksum* number N of the latest remote attestation in [10], P calculates $Fg_1 = Hash(Typ_n || Freq_n || SF_n || Checksum(R_1 \bmod N))$ and $mac_4 = HMAC_{IK}(ID_v || ID_p || R_1 || Fg_1)$.
- Step 8:** Then, P sends the authentication response message $\langle ID_v, ID_p, Fg_1, mac_4 \rangle$ to V .
- Step 9:** After V receives the authentication response message, it first checks mac_4 , then calculates $Fg_2 = Hash(Typ_n || Freq_n || SF_n || Checksum(R_1 \bmod N))$ and verifies whether $Fg_1 = Fg_2$. If true, V would accept the device information of P and initiate the authentication of clock skew. Specifically, V first generates an authentication time interval t . Associated with t , a lower limit is set to successfully detect the clock skew of P , and an upper limit is set to avoid the long authentication time. According to the evaluation results of Arduino boards in our scheme, the value of t is set to $5 \text{ ms} \leq t \leq 200 \text{ ms}$. After t is randomly selected, V generates

the challenge sequence SEQ and calculates $mac_5 = HMAC_{IK}(ID_v || ID_p || SEQ)$. Then, V records the current time Tv_i .

- Step 10:** V sends the clock skew request message $\langle ID_v, ID_p, SEQ, mac_5 \rangle$ to P .
- Step 11:** Upon receiving the clock skew request message, P verifies mac_5 , generates timestamp Tp_i and calculates $mac_6 = HMAC_{IK}(ID_v || ID_p || Tp_i || SEQ + 1)$.
- Step 12:** Then, the node P sends the clock skew response message $\langle ID_v, ID_p, Tp_i, SEQ + 1, mac_6 \rangle$ to V .
- Step 13:** Upon receiving the clock skew response message, V verifies mac_6 and records the timestamp Tp_i , then calculates

$$x_i = Tv_i - Tv_1$$

$$y_i = (Tp_i - Tp_1) - (Tv_i - Tv_1)$$

$$s(n) = \frac{\sum_{i=1}^n (x_i y_i) - n \cdot \bar{x}_n \bar{y}_n}{\sum_{i=1}^n (x_i^2) - n \cdot \bar{x}_n^2}$$

$$\Delta n = |s_n - s_{n-1}|.$$

Here, we define i as the clock skew request count and c_θ as the number of clock skew requests. Only when $i > c_\theta$ is the deviation stable. In this article, we set $c_\theta = 50$. We define θ_1 as the threshold value of the difference between the clock skew and the previous skew. Note that the clock skew is processed using least squares method (LSM). The threshold value is set to $\theta_1 = 2 \times 10^{-6}$ in this article. After the calculation of Δn , if $i > c_\theta$ and $\Delta n < \theta_1$, step 13 is executed. Otherwise, the node V records $S(n)$ and current time T_n . When $T_n - Tv_i \geq t$, step 9 is executed. And we define θ_2 as the threshold value of the difference between the recorded clock skew and the current skew. In this article, θ_2 is set to $\theta_2 = 1 \times 10^{-5}$. The node V verifies whether $|s_p - s(n)| < \theta_2$ according to the previous records of $s(p)$. If true, P is accepted.

VI. SECURITY ANALYSIS

In this section, informal proofs of the security features of CADFA are presented. Furthermore, we verify the security goals of CADFA using the protocol verification tool Tamarin.

A. Informal Security Analysis

1) Mutual Authentication: In the registration stage, the secret shared symmetric key $K = EK || IK$ is preset in V and P . In addition, each step of the authentication will generate and verify the MAC, which not only ensures the integrity but also verifies each party's IK . Besides, the random number is generated and protected by EK during the process, which verifies each party's EK . Considering that there is no secure hardware support in Class-1 IoT devices, chances are that the secret key could be revealed to attackers. To mitigate these kinds of situations, the unclonable device fingerprint is introduced to the authentication procedure. To be specific, it is the clock skew and the verification

node-appointed collection time interval that make it difficult to be forged. In this way, the proof node P and the verification node V are both able to authenticate each other.

2) *Data Confidentiality*: In the authentication process, both parties encrypt confidential information with an encryption key EK to prevent information from being leaked.

3) *Data Integrity*: In the authentication process, both parties use the integrity key IK to protect message integrity, preventing the information from being tampered with.

4) *Resistance to Physical Attacks*: The presented authentication scheme is based on the device fingerprint. This scheme takes advantage of the inevitable deviation in the manufacture of the device clock. Furthermore, during the authentication process, the verification node randomly selects the time interval to collect clock skew, and thus, it is extremely difficult to forge and is resistant to physical attacks.

5) *Resistance to Replay Attacks*: Usually, for most resource-constrained IoT devices in practice, the clock can only keep timing for about 70 min and will be reset to 0 when the upper limit is reached. As a result, using timestamps to resist replay attacks is not applicable. Therefore, our scheme adopts random numbers instead of timestamps to achieve resistance to replay attacks without clock synchronization. Because of the random numbers, whenever a replay attack is encountered, the attack can be easily detected and the subsequent process cannot be performed.

6) *Resistance to Man-in-the-Middle Attacks*: In each step, the node will verify the MAC attached to the message. The MAC is protected by secret key IK , and IK will not be revealed to attackers. In this way, even if the attacker intercepted the message and forged a malicious one using his own $IK_{attacker}$, in the next step the MAC verification will not pass. Thus, our protocol supports resistance to man-in-the-middle attacks.

B. Formal Verification

In this section, we present the formal verification result of the proposed protocol by Tamarin [37]. Tamarin is an up-to-date formal protocol verification tool. It provides general support for modeling protocols, including some built-in functions like asymmetric encryption and hash functions, and cryptographic operators, such as exclusive OR. In Tamarin, protocols can be described in the way of the state's transition by writing rules. A rule defines a state and a set of rules define a transition system.

1) To describe our protocol in Tamarin, we define rules as follows.

a) The rule initialization is the initialization for prove node and verification node.

```
rule Initialization:
[ Fr (~IDv), Fr (~IDp), Fr (~EK), Fr (~IK) ]
--[ Setup() ]->
[ PN_init(SVN, ~IDp, ~IDv, ~EK, ~IK), VN_init(SVN, ~IDp, ~IDv, ~EK, ~IK) ]
```

b) The protocol procedure is defined by rules VNs and PNs. VNs describe the verification node operations, and PNs describe the prove node operations. During each step, the receiver will check the sender ID and verify the MAC message.

```
rule VN1:
let
  C1 = senc (~R1, EK)
  mac1 = HMAC(<C1, IDp>, IK)
in
[ VN_init(SVN, IDp, IDv, EK, IK), Fr (~R1) ]
--[ Sendmac1(SVN, mac1), SecMsg(~R1) ]->
[ VN_store(SVN, IDp, IDv, EK, IK, ~R1), Out(<IDv, IDp, C1, mac1>) ]

rule PN1:
let
  R11 = sdec(C1, EK1)
  mac11 = HMAC(<C1, IDp1>, IK1)
  C2 = senc(<R11, ~R2>, EK1)
  mac2 = HMAC(<C2, IDv1, IDp1>, IK1)
in
[ PN_init(SVN, IDp1, IDv1, EK1, IK1), In(<IDp, IDv, C1, mac1>), Fr (~R2) ]
--[ Recvmac1(SVN, mac1), Eq(IDv, IDv1), Eq(mac11, mac1), Sendmac2(SVN, mac2), SecMsg(R11) ]->
[ PN_store(SVN, IDp1, IDv1, EK1, IK1, R11, ~R2), Out(<IDp1, mac2, C2>) ]

rule VN2:
let
  R21 = snd(sdec(C2, EK))
  mac21 = HMAC(<C2, IDv, IDp1>, IK)
  mac3 = HMAC(<R21, IDv, IDp1>, IK)
in
[ VN_store(SVN, IDp, IDv, EK, IK, R1), In(<IDp1, mac2, C2>) ]
--[ Eq(IDp, IDp1), Eq(mac2, mac21), Recvmac2(SVN, mac2), SecMsg(R1), SecMsg(R1), Sendmac3(SVN, mac3) ]->
[ VN_store2(SVN, IDp, IDv, EK, IK, R1, R21), Out(<IDv, R21, mac3>) ]

rule PN2:
let
  mac31 = HMAC(<R2, IDv, IDp1>, IK1)
  Fg1 = h(~DF1)
  mac4 = HMAC(<IDv, IDp1, R11, R2, Fg1>, IK1)
in
[ PN_store(SVN, IDp1, IDv1, EK1, IK1, R11, R2), In(<mac3, IDv, R21>), Fr (~DF1) ]
--[ Eq(IDv, IDv1), Eq(R2, R21), SecMsg(R11), Eq(mac3, mac31), Recvmac3(SVN, mac3), Sendmac4(SVN, mac4) ]->
[ PN_store2(SVN, IDp1, IDv1, EK1, IK1, R11, R2, Fg1), Out(<IDp1, Fg1, mac4>) ]

rule VN3:
let
  mac41 = HMAC(<IDv, IDp1, R1, R21, Fg1>, IK)
  mac5 = HMAC(<IDv, IDp1, ~SEQ>, IK)
in
[ VN_store2(SVN, IDp, IDv, EK, IK, R1, R21), In(<IDp1, Fg1, mac4>), Fr (~Tvi), Fr (~SEQ) ]
--[ Eq(IDp, IDp1), Eq(mac4, mac41), SecMsg(R1), Recvmac4(SVN, mac4), Sendmac5(SVN, mac5) ]->
[ VN_store3(SVN, IDp, IDv, EK, IK, R1, R21, ~Tvi, ~SEQ), Out(<IDv, ~SEQ, mac5>) ]

rule PN3:
let
  mac51 = HMAC(<IDv, IDp1, SEQ>, IK1)
  mac6 = HMAC(<IDv, IDp1, ~Tpi, h(SEQ)>, IK1)
in
[ PN_store2(SVN, IDp1, IDv1, EK1, IK1, R11, R2, Fg1), In(<IDv, SEQ, mac5>), Fr (~Tpi) ]
--[ Eq(IDv, IDv1), Eq(mac5, mac51), SecMsg(R11), Recvmac5(SVN, mac5), Sendmac6(SVN, mac6) ]->
[ Out(<IDp1, mac6, h(SEQ), ~Tpi>) ]

rule VN4:
let
  mac61 = HMAC(<IDv, IDp1, Tpi, h(SEQ)>, IK)
in
[ VN_store3(SVN, IDp, IDv, EK, IK, R1, R21, ~Tvi, SEQ), In(<IDp1, mac6, h(SEQ), Tpi>)]
--[ Eq(IDp, IDp1), Eq(mac6, mac61), Recvmac6(SVN, mac6), SecMsg(R1) ]->
[ ]
```

2) To verify the security properties of our authentication protocol, we define the following lemmas.

TABLE II
SECURITY COMPARISON

	Lanze's scheme [27]	Zuo's scheme [16]	Zhou's scheme [33]	Our method
Mutual identity authentication	No	Yes	No	Yes
Data confidentiality	No	Yes	No	Yes
Data integrity	No	Yes	No	Yes
Resistance to physical attacks	Yes	Yes	Yes	Yes
Resistance to replay attacks	Yes	Yes	Yes	Yes
Resistance to man-in-the-middle attacks	Yes	Yes	Yes	Yes
Formal verification	No	No	No	Yes

```

/* All well-formedness checks were successful. */
end

=====
summary of summaries:

analyzed: CADFA0901.spthy

  Executablemac1 (exists-trace): verified (7 steps)
  Executablemac2 (exists-trace): verified (11 steps)
  Executablemac3 (exists-trace): verified (14 steps)
  Executablemac4 (exists-trace): verified (15 steps)
  Executablemac5 (exists-trace): verified (18 steps)
  Executablemac6 (exists-trace): verified (21 steps)
  PN_auth_VN (all-traces): verified (185 steps)
  VN_auth_PN (all-traces): verified (26 steps)
  Secrecy (all-traces): verified (9896 steps)

=====

```

Fig. 4. Tamarin verification results.

a) Executable guarantees that each step is actually executed.

```

lemma Executablemac1:
  exists-trace
  "Ex VN msg #a #b. Sendmac1(VN,msg)@a & Recvmac1(VN,msg)@b"

lemma Executablemac2:
  exists-trace
  "Ex VN msg #a #b. Recvmac2(VN,msg)@a & Sendmac2(VN,msg)@b"

lemma Executablemac3:
  exists-trace
  "Ex VN msg #a #b. Recvmac3(VN,msg)@a & Sendmac3(VN,msg)@b"

lemma Executablemac4:
  exists-trace
  "Ex VN msg #a #b. Recvmac4(VN,msg)@a & Sendmac4(VN,msg)@b"

lemma Executablemac5:
  exists-trace
  "Ex VN msg #a #b. Recvmac5(VN,msg)@a & Sendmac5(VN,msg)@b"

lemma Executablemac6:
  exists-trace
  "Ex VN msg #a #b. Recvmac6(VN,msg)@a & Sendmac6(VN,msg)@b"

```

b) Secrecy guarantees the confidentiality of secret messages.

```

lemma Secrecy:
  "All n #a. SecMsg(n) @a ==>(not(Ex #b. K(n) @b))"

```

c) The lemma auth guarantees mutual authentication between PN and VN.

```

lemma PN_auth_VN:
  "( All VN msg #a. Recvmac1(VN, msg) @ #a
  ==>
  ( (Ex #b. Sendmac1(VN, msg) @b) )"

lemma VN_auth_PN:
  "( All VN msg #a. Recvmac2(VN,msg) @ #a
  ==>
  ( (Ex #b. Sendmac2(VN,msg) @b) )"

```

As shown in Fig. 4, the verification results indicate our proposed protocol achieves the confidentiality of secret messages

and mutual authentication. Confidentiality indicates the messages will not be revealed. The auth lemmas indicate mutual authentication and man-in-the-middle attack resistance.

Finally, we compare all of the above security properties with related protocols in Table II. As shown in Table II, all the methods can achieve common attack resistance because of the use of fingerprints. Except for our scheme, none of the related methods uses formal verification. In addition, the schemes in [27] and [33] cannot achieve mutual authentication, data confidentiality, and integrity protection.

VII. PERFORMANCE ANALYSIS

In this section, we present CADFA performance in terms of time overhead and bandwidth overhead.

A. Time Overhead

In our protocol, the major time consumption includes the time interval t , the computation time of the proof node, and the communication overhead. Considering the advanced performance of the verification node, the computation cost of which is not included. Here, we set the time interval as $5 \text{ ms} \leq t \leq 200 \text{ ms}$. The simulation result on the Arduino boards shows that the computation time of AES and HMAC (SHA-256) is 0.9 and 12.8 ms, respectively. And the communication time between the proof node and the verification node is 4.1 ms. With the increase of t , the computation can be performed during the request interval. Thus, the total time cost is calculated as

$$\begin{cases} 73.1 + 29.79 \cdot n, t \leq 29.7 \\ 73.1 + t \cdot n, t > 29.7 \end{cases}$$

where n represents the number of requests, and t represents the time interval between two requests. We compare our authentication scheme with Lanze [27], Zuo [16], and Zhou [33] in Fig. 5. As shown in Fig. 5, our scheme outperforms other related schemes and the advantage grows with the increasing number of requests. It is because of the active requesting mechanism instead of waiting for data packets that greatly reduces the collection time consumption. The attribute-based scheme in [16] depends on a large amount of computation, which does not apply to resource-constrained Class-1 IoT devices. The scheme proposed by Zhou [33] requires a long time to collect RF data.

B. Bandwidth Overhead

We assume the length of the generated random number is 4 B, that of a timestamp is 2 B, and that of the device identifier is 2 B. In addition, SHA-256 is selected when calculating MAC, which

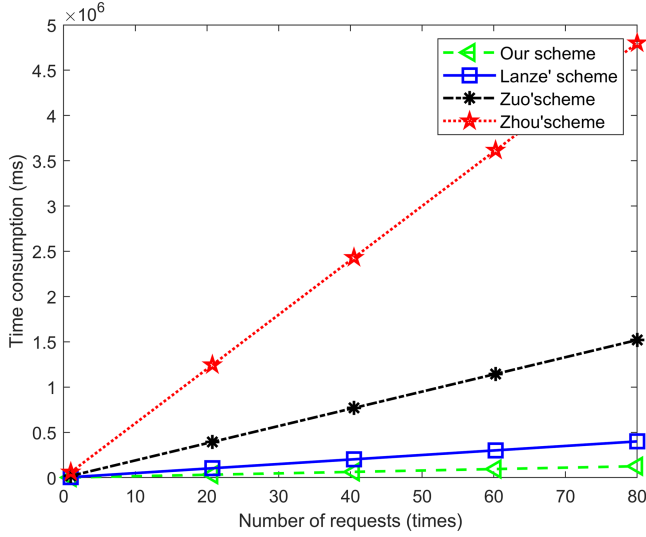


Fig. 5. Comparison of time overhead.

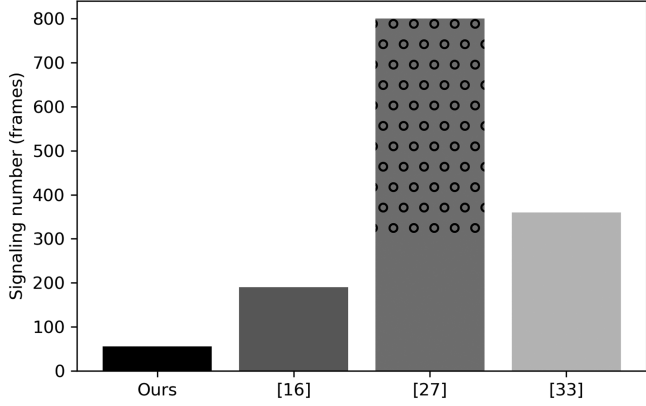


Fig. 6. Signaling overhead comparison.

generates a 32-B length message. Thus, the total bandwidth is $188 + 78 \cdot n$ B, where n represents the number of requests. Under the circumstance that $n \geq 50$ in this article, 4088 B will be consumed for one complete clock skew authentication.

C. Signaling Overhead

In this article, we extract the device fingerprint through communication messages. In order to obtain a stable slope value, a threshold of the communication rounds needs to be achieved, which is c_θ we defined in this article. When c_θ is set to 50, the total rounds are $6+50 = 56$ rounds. It is acceptable compared with related schemes as shown in Fig. 6. The scheme in [16] takes approximately 96 h to finish the analysis with 18 166 fingerprints. On average, each fingerprint needs to take $96 \cdot 60 \cdot 60 / 18166 = 19$ s. If the time interval is 100 ms as mentioned in [16], they need 190 rounds for each fingerprint, which is much larger than 56 of our scheme. Lanze et al. mentioned that their method needs approximately 800 and 300 beacon frames in [27]. It is clear that 300 rounds of the scheme in [27] are still larger than 56 of our scheme. The scheme in [33]

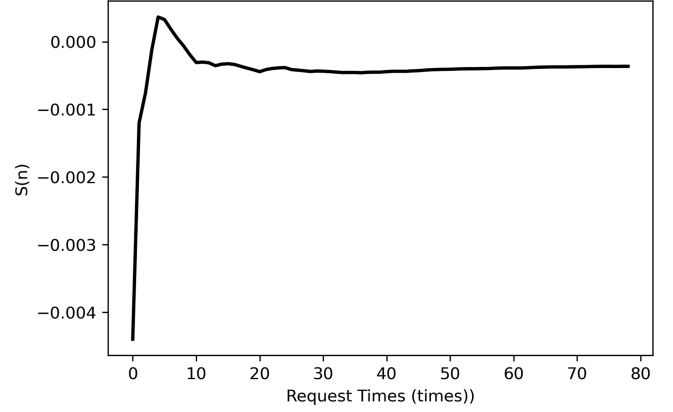


Fig. 7. Clock skew data.

needs at least 360 frames to extract the fingerprint used for identification, which is also larger than 56 of our scheme.

In this case, we believe that the computation overhead, communication bandwidth, and signaling overhead are acceptable compared with related schemes.

VIII. SIMULATION EVALUATION

To evaluate the proposed scheme, we run our protocol on a small cluster of Arduino Mega boards. To begin with, we determine the values of the arguments in our protocol.

First, we define the values of c_θ and θ_1 . We develop our protocol in C language using a CPU-level method with microsecond-level accuracy and get the clock skew data. Fig. 7 shows the clock skew data with the increasing number of requests during the authentication procedure. The horizontal coordinate indicates the number of measurements, and the vertical coordinate indicates the clock skew value. As shown in Fig. 7, after 50 times, the curve becomes convergent and the difference between the 49th and 50th is only 1.027×10^{-6} . So the minimum number of requests of clock skew is set to $c_\theta = 50$, the threshold value of the ending request is set to $\theta_1 = 2 \times 10^{-6}$, and the threshold value of the difference between the final skew and original registration skew is set to $\theta_2 = 1 \times 10^{-5}$.

Second, we determine the value of θ_2 . Here, we define *clustersimilarity* representing the percentage of the devices of the same clock skew value. A smaller *clustersimilarity* value indicates more devices can be distinguished. The 0% *clustersimilarity* represents the unique existence of one skew and 100% represents all the devices are the same. The calculation of *clustersimilarity* is as follows. For a cluster consisting of n devices, collect n clock skew data $Sn_1, Sn_2, \dots, Sn_n$. For each point, calculate $a_{ij} = |Sn_i - Sn_j|$, $i, j \in [1, n]$, and $i \neq j$. If $a_{ij} < \theta_2$, then $Sum = Sum + 1$. In the end, *clustersimilarity* = Sum/C_n .

The experiment result is shown in Table III. We cannot distinguish any individual device when $\theta_2 \geq 0.002$. As the θ_2 decreases, the *clustersimilarity* reduces. When θ_2 reaches 1×10^{-5} , each device can be distinguished.

At this point, all the arguments are determined, and we simulate our protocol on a small cluster of IoT devices based on the

TABLE III
DEVICE SIMILARITY

θ_2	0.002	0.001	0.0005	0.0001	0.00005	0.00003	0.00002	0.00001
Similarity	100%	85.71%	71.43%	25.00%	14.29%	7.14%	3.57%	0.00%

TABLE IV
FINGERPRINT DATA OF EACH DEVICE

No.	1	2	3	4	5	6	7	8
Interval								
5ms	-0.000773627	-0.00033533	0.000661049	-0.000414218	-0.000460822	-0.000310266	-0.000211079	-0.000349523
10ms	-0.000773627	-0.00033533	0.000661049	-0.000414218	-0.000460822	-0.000310266	-0.000211079	-0.000349523
50ms	-0.000773627	-0.00033533	0.000661049	-0.000414218	-0.000460822	-0.000310266	-0.000211079	-0.000349523
100ms	-0.000773627	-0.00033533	0.000661049	-0.000414218	-0.000460822	-0.000310266	-0.000211079	-0.000349523
200ms	-0.000773627	-0.00033533	0.000661049	-0.000414218	-0.000460822	-0.000310266	-0.000211079	-0.000349523

above-determined parameters. In this article, we use Arduino boards as verification nodes to construct a small IoT device cluster. They can communicate with each other through a serial port where the baud rate is set to 115 200. We ran our protocol on each of the devices 80 times and collected the clock skew data as shown in Table IV. The device fingerprint data shows that each one's device fingerprint is unique. Thus, we can achieve a 100% accurate rate under the proper arguments definition.

IX. CONCLUSION

To verify the identity of remote devices more than integrity, we proposed a clock skew-based active IoT device fingerprint authentication scheme. We improve the current clock skew-based remote attestation protocol by introducing device fingerprint and replacing passive collecting with active requesting, which greatly increases authentication efficiency. The informal security analysis and formal verification show that our scheme achieves mutual authentication, and resists malicious attacks including man-in-the-middle attacks, replay attacks, counterfeit attacks, etc. Besides, the evaluation result on the Arduino boards indicates that our protocol performs well on Class-I IoT devices. The limitation of the proposed scheme is that with the authentication efficiency increases, the communication bandwidth overhead increases linearly. In our future work, we will optimize the fingerprint extraction process, reduce the communication rounds, and simplify the transmission frames to decrease bandwidth and signaling overhead.

REFERENCES

- [1] C. Bormann, M. Ersue, and A. Keranen, "Terminology for constrained-node networks," Internet Eng. Task Force, Tech. Rep. RFC 7228, May 2014. [Online]. Available: <http://www.ietf.org/rfc/rfc7228.txt>
- [2] S. Yoon, J. Kim, and Y. Jeon, "Security considerations based on classification of IoT device capabilities," in *Proc. 9th Int. Conf. Adv. Serv. Comput.*, 2017, pp. 1–3.
- [3] C. Kil, E. C. Sezer, A. M. Azab, P. Ning, and X. Zhang, "Remote attestation to dynamic system properties: Towards providing complete system integrity evidence," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2009, pp. 115–124.
- [4] B. Parno, J. M. McCune, and A. Perrig, "Bootstrapping trust in commodity computers," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 414–429.
- [5] M. Ammar, B. Crispo, and G. Tsudik, "Simple: A remote attestation approach for resource-constrained IoT devices," in *Proc. IEEE/ACM 11th Int. Conf. Cyber-Phys. Syst.*, 2020, pp. 247–258.
- [6] N. A. Ali, M. Drieberg, and P. Sebastian, "Deployment of micaz mote for wireless sensor network applications," in *Proc. IEEE Int. Conf. Comput. Appl. Ind. Electron.*, 2011, pp. 303–308.
- [7] N. Matthys, F. Yang, W. Daniels, W. Joosen, and D. Hughes, "Demonstration of micropnp: The zero-configuration wireless sensing and actuation platform," in *Proc. IEEE 13th Annu. Int. Conf. Sens., Commun., Netw.*, 2016, pp. 1–2.
- [8] A. Seshadri, A. Perrig, L. Van Doorn, and P. Khosla, "Swatt: Software-based attestation for embedded devices," in *Proc. IEEE Symp. Secur. Privacy*, 2004, pp. 272–282.
- [9] X. Kovah, C. Kallenberg, C. Weathers, A. Herzog, M. Albin, and J. Butterworth, "New results for timing-based attestation," in *Proc. IEEE Symp. Secur. Privacy*, 2012, pp. 239–253.
- [10] J. Cao, T. Zhu, R. Ma, Z. Guo, Y. Zhang, and H. Li, "A software-based remote attestation scheme for Internet of Things devices," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1422–1434, Mar./Apr. 2022.
- [11] J. Kong, F. Koushanfar, P. K. Pendyala, A.-R. Sadeghi, and C. Wachsmann, "Pufatt: Embedded platform attestation based on novel processor-based pufs," in *Proc. 51st Annu. Des. Automat. Conf.*, 2014, pp. 1–6.
- [12] S. Aneja, N. Aneja, and M. S. Islam, "IoT device fingerprint using deep learning," in *Proc. IEEE Int. Conf. Internet Things Intell. Syst.*, 2018, pp. 174–179.
- [13] A. Ibrahim, A.-R. Sadeghi, G. Tsudik, and S. Zeitouni, "Darpa: Device attestation resilient to physical attacks," in *Proc. 9th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2016, pp. 171–182.
- [14] P. Eckersley, "How unique is your web browser?," in *Proc. Privacy Enhancing Technol.: 10th Int. Symp.*, 2010, pp. 1–18.
- [15] A. Vastel, P. Laperdrix, W. Rudametkin, and R. Rouvoy, "Fp-stalker: Tracking browser fingerprint evolutions," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 728–741.
- [16] C. Zuo, H. Wen, Z. Lin, and Y. Zhang, "Automatic fingerprinting of vulnerable ble IoT devices with static uuids from mobile apps," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1469–1483.
- [17] D. S. Boning and J. E. Chung, "Statistical metrology: Understanding spatial variation in semiconductor manufacturing," in *Proc. SPIE Int. Soc. Opt. Eng.*, vol. 2874, pp. 16–26, 1996.
- [18] K. A. Bowman, S. G. Duvall, and J. D. Meindl, "Impact of die-to-die and within-die parameter fluctuations on the maximum clock frequency distribution for gigascale integration," *IEEE J. Solid-State Circuits*, vol. 37, no. 2, pp. 183–190, Feb. 2002.
- [19] S. R. Nassif, "Modeling and forecasting of manufacturing variations (embedded tutorial)," in *Proc. Asia South Pacific Des. Automat. Conf.*, 2001, pp. 145–150.
- [20] B. Gassend, D. Clarke, M. Van Dijk, and S. Devadas, "Silicon physical random functions," in *Proc. 9th ACM Conf. Comput. Commun. Secur.*, 2002, pp. 148–160.
- [21] J. W. Lee, D. Lim, B. Gassend, G. E. Suh, M. Van Dijk, and S. Devadas, "A technique to build a secret key in integrated circuits for identification and authentication applications," in *Proc. IEEE Symp. VLSI Circuits. Dig. Tech. Papers*, 2004, pp. 176–179.

- [22] R. Pappu, B. Recht, J. Taylor, and N. Gershenfeld, "Physical one-way functions," *Science*, vol. 297, no. 5589, pp. 2026–2030, 2002.
- [23] A. Das, N. Borisov, and M. Caesar, "Tracking mobile web users through motion sensors: Attacks and defenses," in *Proc. NDSS*, 2016, pp. 1–5.
- [24] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Trans. Dependable Secure Comput.*, vol. 2, no. 2, pp. 93–108, Apr.–Jun. 2005.
- [25] S. Jana and S. K. Kasera, "On fast and accurate detection of unauthorized wireless access points using clock skews," in *Proc. 14th ACM Int. Conf. Mobile Comput. Netw.*, 2008, pp. 104–115.
- [26] S. Sharma, A. Hussain, and H. Saran, "Experience with heterogeneous clock-skew based device fingerprinting," in *Proc. Workshop Learn. Authoritative Secur. Exp. Results*, 2012, pp. 9–18.
- [27] F. Lanze, A. Panchenko, B. Braatz, and A. Zinnen, "Clock skew based remote device fingerprinting demystified," in *Proc. IEEE Glob. Commun. Conf.*, 2012, pp. 813–819.
- [28] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 5, pp. 519–532, Sep. 2015.
- [29] I. Sanchez-Rola, I. Santos, and D. Balzarotti, "Clock around the clock: Time-based device fingerprinting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1502–1514.
- [30] F. Lorenz et al., "Fingerprinting analog iot sensors for secret-free authentication," in *Proc. 29th Int. Conf. Comput. Commun. Netw.*, 2020, pp. 1–6.
- [31] H. Jafari, O. Omotere, D. Adesina, H.-H. Wu, and L. Qian, "IoT devices fingerprinting using deep learning," in *Proc. IEEE MILCOM Mil. Commun. Conf.*, 2018, pp. 1–9.
- [32] Y. Cheng, X. Ji, J. Zhang, W. Xu, and Y.-C. Chen, "Demicpu: Device fingerprinting with magnetic signals radiated by CPU," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1149–1170.
- [33] X. Zhou, A. Hu, G. Li, L. Peng, Y. Xing, and J. Yu, "A robust radio-frequency fingerprint extraction scheme for practical device recognition," *IEEE Internet Things J.*, vol. 8, no. 14, pp. 11276–11289, Jul. 2021.
- [34] A. Ibrahim, A.-R. Sadeghi, and S. Zeitouni, "Seed: Secure non-interactive attestation for embedded devices," in *Proc. 10th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2017, pp. 64–74.
- [35] F. Kohnhäuser, N. Büscher, S. Gabmeyer, and S. Katzenbeisser, "Scapi: A scalable attestation protocol to detect software and physical attacks," in *Proc. 10th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2017, pp. 75–86.
- [36] A. Ibrahim, A.-R. Sadeghi, and G. Tsudik, "US-aid: Unattended scalable attestation of IoT devices," in *Proc. IEEE 37th Symp. Reliable Distrib. Syst.*, 2018, pp. 21–30.
- [37] T. Team, "Tamarin-prover manual—security protocol analysis in the symbolic model," Tech. Rep. Version of 2018-11-27, 2016. [Online]. Available: <https://tamarin-prover.github.io/manual>



Chao Shang received the B.S. degree in information security in 2020 from Xidian University, Xian, China, where he is currently working toward the Ph.D. degree in cyber security with the School of Cyber Engineering.

His research interests include authentication protocol, device fingerprint, and intelligent vehicle security.



Jin Cao (Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2015.

He is currently a Professor with the School of Cyber Engineering, Xidian University. His research interests include wireless network security and LTE/LTE-A/5-G networks.



Tong Zhu received the M.Sc. degree in cyber security from Xidian University, Xi'an, China, in 2022.

His current research interest includes IoT security.



Yinghui Zhang (Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2013.

He is currently a Professor with the National Engineering Laboratory for Wireless Security, Xi'an University of Posts and Telecommunications, Xi'an. His current research interests include wireless network security, cloud computing security, access control, security and privacy in IoT, etc.



Ben Niu (Member, IEEE) received the B.S. degree in information security, and the M.S. and Ph.D. degrees in cryptography from Xidian University, Xian, China, in 2006, 2010, and 2014, respectively.

He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. From 2011 to 2013, he was a Visiting Scholar with Pennsylvania State University, Pennsylvania, PA, USA. His current research interests include network security and privacy computing.



Hui Li (Member, IEEE) received the B.Sc. degree in radio electronics from Fudan University, Shanghai, China, in 1990, and the M.S. and Ph.D. degrees in telecommunications and information system from Xidian University, Xi'an, China, in 1993 and 1998, respectively.

Since 2005, he has been a Professor with the School of Cyber Engineering, Xidian University. He has coauthored two books. His research interests include cryptography, wireless network security, information theory, and network coding.

Dr. Li served as Technique Committee Co-Chairs of ISPEC 2009 and IAS 2009.